

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINIPROJECT

FINAL PROJECT DOCUMENTATION

Electronics Store Inventory Management System

TechZone Electronics | Group 9

Document Item	Details
Document Package	SRS + Software Architecture & Design Specification + Software Test Plan
Version	1.1
Project Type	Software Engineering – Level 3 Mini-Project
Case Study	TechZone Electronics (Fictional Electronics Retail Store)
Development Model	Agile / Incremental Sprints
Technology Stack	React + JavaScript / Django REST Framework + Python / PostgreSQL
Date	03 October 2026
Group	Team 9

Team Members

SRN	Student	Primary Functional Ownership
PES1UG24CS068	Annamalai N	User Authentication & Role Management
PES1UG24CS077	Apurv Kumar Singh	Product & Inventory Management
PES1UG24CS091	Atharv Mittal (Team Lead)	Supplier & Purchase Management
PES1UG24CS099	Avaneesh H D	Sales & Reporting Management

Contents

1. Part I — Software Requirements Specification (previously completed SRS, included in full)
2. Part II — Software Architecture & Design Specification (SAD)
3. Part III — Software Test Plan (STP)
4. Appendix — Submission and Repository Checklist

PART I – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

PES UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE REQUIREMENTS
SPECIFICATION

Electronics Store Inventory Management System

TechZone Electronics | Group 9

Document Item	Details
Version	1.0
Prepared For	Software Engineering – Level 3 Mini-Project
Case Study	TechZone Electronics (Fictional Electronics Retail Store)
Development Model	Agile / Incremental Sprints
Prepared By	Team 9
Date	September 2026

Team Members

Sl. No.	USN	Student Name	Primary Functional Ownership
1	PES1UG24CS068	Annamalai N	User Authentication & Role Management
2	PES1UG24CS077	Apurv Kumar Singh	Product & Inventory Management
3	PES1UG24CS091	Atharv Mittal (Team Lead)	Supplier & Purchase Management
4	PES1UG24CS099	Avaneesh H D	Sales & Reporting Management

Table of Contents

1. 1. Introduction and Purpose
2. 2. Product Scope
3. 3. Intended Users
4. 4. Product Functions
5. 5. User Classes
6. 6. Operating Environment
7. 7. Constraints and Assumptions
8. 8. External Interface Requirements
9. 9. Use Case / Analysis Models
10. 10. Functional Requirements
11. 11. Non-Functional Requirements
12. 12. Security Requirements
13. 13. Business / Domain Rules
14. 14. Database / Field Requirements
15. 15. Glossary
16. 16. Requirement Traceability Matrix

1. Introduction and Purpose

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Electronics Store Inventory Management System proposed for TechZone Electronics, a fictional small-to-medium electronics retail shop used as the project case study. The system is a web-based internal application for managing products, stock, suppliers, purchases, sales, user access, and operational reports.

The SRS provides a common reference for the development team, faculty/evaluators, testers, and future maintainers. It converts project objectives into clear, uniquely identified, measurable, and testable requirements.

1.2 Intended Audience and Reading Suggestions

- Project team members – requirements, ownership, design and implementation basis.
- Faculty/evaluators – completeness, feasibility, clarity and traceability assessment.
- Developers – architecture, API, database and UI implementation.
- Testers – derivation of unit, integration, system and acceptance tests.
- Maintainers – understanding intended behavior and business rules.

Sections 1–7 provide context; Sections 8–10 define interfaces, analysis models and functional requirements; Sections 11–13 define quality, security and business rules; Sections 14–16 define data, terminology and traceability.

1.3 Requirement Conventions

- FR-xxx = functional requirement; NFR-xxx = non-functional requirement; SEC-xxx = security requirement; BR-xxx = business/domain rule; UC-xx = use case.
- “Shall” denotes mandatory behavior.
- Planned architecture, code and test references in the RTM are placeholders to be updated after implementation.

1.4 References

Reference	Description
PES University SE Mini-Project Guidelines, 2026	Faculty-provided project and SRS guidance/template.
Team 9 Synopsis – Electronics Store Inventory Management System	Project proposal defining problem, scope, users, features, ownership, plan and technology stack.
React / Django REST Framework / PostgreSQL documentation	Technical references for the planned implementation stack.

2. Product Scope

The system will centralize internal inventory and transaction operations of TechZone Electronics. It will maintain consistent product and stock records while connecting purchasing and sales events to inventory changes.

2.1 In-Scope

- Authentication, user management and role-based access.
- Product/category management and product search.
- Inventory quantity tracking, adjustments, low-stock identification and inventory history.
- Supplier management and purchase order management.
- Recording received purchases and increasing stock accordingly.

- Sales transactions and inventory reduction.
- Operational reports for inventory, purchases, sales and low-stock items.
- Validation, filtering and common error handling.

2.2 Out-of-Scope

- Public e-commerce storefront and online ordering.
- Online payment gateway integration.
- Delivery/courier management.
- Full accounting, payroll, taxation or ERP functions.
- AI demand forecasting.
- Multi-store enterprise management.
- Customer loyalty/CRM and marketing automation.
- Supplier payment settlement or banking integration.

3. Intended Users

User / Stakeholder	Primary Need	Typical Activities
Store Administrator	Control access and master data	Users/roles, products, suppliers, inventory controls, reports
Inventory / Store Staff	Maintain accurate stock	Product search, stock status, permitted adjustments, low-stock review
Purchase Staff	Manage procurement	Suppliers, purchase orders, receipts, purchase history
Sales Staff	Record retail sales	Product availability, sale entry, transaction history
Store Manager / Owner	Monitor operations	Inventory, sales, purchases and low-stock reports
Developers / Testers	Build and validate	Implementation, testing, reviews and documentation

4. Product Functions

ID	Major Function	Summary	Primary Owner
F-01	Authentication & Roles	Login, logout, users, roles and permissions.	Annamalai N
F-02	Product & Category	Product/category creation, update, search and maintenance.	Apurv Kumar Singh
F-03	Inventory	Stock quantities, adjustments, low-stock status and history.	Apurv Kumar Singh
F-04	Supplier Management	Supplier profile and status management.	Atharv Mittal
F-05	Purchase Management	Purchase orders, receipts and stock increases.	Atharv Mittal
F-06	Sales Management	Sales transactions, validation and stock decreases.	Avaneesh H D
F-07	Reporting	Inventory, purchase, sales and low-stock reports.	Avaneesh H D
F-08	Search & Filtering	Search/filter products, suppliers and transactions.	Shared

5. User Classes

5.1 Administrator

Highest-privilege operational user. Manages users/roles and has broad access needed to administer store data.

5.2 Inventory Staff

Maintains product and stock information, views availability, performs permitted adjustments and monitors low stock.

5.3 Purchase Staff

Maintains suppliers, creates purchase records and records receipt of goods.

5.4 Sales Staff

Records sales and needs access to product availability and transaction history.

5.5 Manager / Owner

Primarily reviews operational reports, inventory status, purchases and sales.

6. Operating Environment

Component	Planned Environment
Frontend	React with JavaScript; responsive browser-based UI
Backend	Python with Django REST Framework
Database	PostgreSQL
Client	Current stable Chrome/Chromium, Firefox or Edge on desktop/laptop
Server	Linux-based development/deployment environment
Tools	Git/GitHub, Jira, Docker, Jenkins, SonarQube
Testing	pytest plus API/integration/system testing
Network	LAN/Wi-Fi for development; HTTPS-enabled deployment for production

The application is intended to operate using standard web-browser and server infrastructure without specialized client hardware.

7. Constraints and Assumptions

7.1 Design and Implementation Constraints

- React/JavaScript, Django REST Framework/Python and PostgreSQL shall be used as the primary stack.
- Client-server web architecture shall be used.
- Git/GitHub and Jira shall be used for source control and project tracking.
- Role authorization shall be enforced at the backend/API level.
- Database referential integrity shall be maintained.
- The project is an academic mini-project; integrations outside the stated scope are not required.

7.2 Assumptions and Dependencies

- Authorized staff have valid credentials and a supported browser.
- Initial master data is entered by authorized staff.
- Stock is maintained in integer units.
- A sale cannot be completed when stock is insufficient.
- Received purchases increase inventory only after successful receipt recording.
- PostgreSQL, backend API and required deployment services are available.
- Approved requirement changes will be reflected through SRS revision control.

8. External Interface Requirements

8.1 User Interfaces

- Login screen with credential validation and session/logout behavior.
- Role-appropriate dashboard and navigation.
- Product/inventory tables and forms with search, filters and validation.
- Supplier/purchase screens with supplier selection, item entry, totals, status and receipt actions.
- Sales screen with product search, quantity entry, stock validation, totals and confirmation.
- Reports screen with filters and tabular summaries.
- Consistent labels, required-field indicators, success messages and error messages.
- Unauthorized actions shall be rejected by the backend even if a client request is manually constructed.

8.2 Software Interfaces

Interface	Direction	Purpose / Data
React ↔ Django REST API	Bidirectional	JSON requests/responses for application modules.
Django ↔ PostgreSQL	Bidirectional	Persistent application data through the database layer/ORM.
Git/GitHub	Outbound	Source control and collaboration.
Jenkins	Inbound/Outbound	Automated build/test pipeline.
SonarQube	Outbound	Static analysis and quality checks.

8.3 Communications Interfaces

- Frontend/backend communication shall use HTTP/HTTPS REST APIs.
- JSON shall be the primary data format.
- Production communication shall use HTTPS.
- API responses shall use appropriate HTTP status codes.
- Database connections shall remain internal and shall not be exposed directly to browsers.

9. Use Case / Analysis Models

9.1 Use Case Catalogue

ID	Use Case	Primary Actor	Goal
UC-01	Authenticate User	All users	Securely access the system according to role.
UC-02	Manage Users and Roles	Administrator	Create/update/deactivate users and assign roles.
UC-03	Manage Products	Admin / Inventory	Maintain product and category data.

ID	Use Case	Primary Actor	Goal
UC-04	Manage Inventory	Inventory Staff	View, adjust and monitor stock.
UC-05	Manage Suppliers	Purchase Staff / Admin	Maintain supplier records.
UC-06	Create Purchase Order	Purchase Staff	Record procurement from a supplier.
UC-07	Receive Purchase	Purchase Staff	Record received items and increase stock.
UC-08	Record Sale	Sales Staff	Record a sale and decrease stock.
UC-09	View Reports	Manager / Admin	Review inventory, purchase, sales and low-stock data.
UC-10	Search and Filter	Authorized users	Locate relevant records efficiently.

9.2 Main Business Flow

Authenticate User → Product/Inventory Setup → Supplier & Purchase Processing → Inventory Increase on Receipt → Sales Processing → Inventory Decrease on Sale → Reports and Low-Stock Monitoring.

9.3 Use Case Details

UC-01 Authenticate User

Main flow: Enter valid credentials → validate → establish authenticated session/token → show role-specific dashboard.

Exception handling: Invalid credentials or inactive account → deny access.

UC-02 Manage Users and Roles

Main flow: Admin opens user management → enters data → assigns role → validates → saves.

Exception handling: Duplicate account data or invalid role → reject.

UC-03 Manage Products

Main flow: Open products → create/update category/product → validate → save.

Exception handling: Duplicate SKU or invalid values → reject.

UC-04 Manage Inventory

Main flow: View stock → enter permitted adjustment and reason → validate → save transaction.

Exception handling: Invalid quantity or negative resulting stock → reject.

UC-05 Manage Suppliers

Main flow: Add/update supplier → validate required details → save.

Exception handling: Missing/invalid required data → reject.

UC-06 Create Purchase Order

Main flow: Select supplier → add items/quantities → calculate total → save order.

Exception handling: Inactive supplier or invalid item/quantity → reject.

UC-07 Receive Purchase

Main flow: Select purchase → confirm received quantities → validate → record receipt → increase stock.

Exception handling: Received quantity beyond allowed amount → reject.

UC-08 Record Sale

Main flow: Select products → enter quantities → validate stock → calculate total → complete sale → reduce stock.

Exception handling: Insufficient stock → reject and do not create sale.

UC-09 View Reports

Main flow: Select report/filter → retrieve and aggregate data → display results.

Exception handling: Invalid filter range → validation message.

UC-10 Search and Filter

Main flow: Enter search/filter → retrieve matches → display results/empty state.

Exception handling: No matches → show empty state.

9.4 Core Entity Analysis

Entity	Key Relationships
User / Role	A user has one role; the role controls permissions.
Category / Product	A category can contain multiple products; each product references a category.
Supplier / PurchaseOrder	A supplier can have multiple purchase orders; each purchase order references one supplier.
PurchaseOrder / PurchaseItem / Product	Each purchase contains one or more items; each item references a product and quantity.
Sale / SaleItem / Product	Each sale contains one or more items; each item references a product and quantity.
Product / InventoryTransaction	Inventory transactions record stock changes for products.

The final ER/UML diagrams should be added to the design documentation once the physical schema is frozen. Section 14 is the initial logical data model.

10. Functional Requirements

Requirements are grouped by system feature. Each is uniquely identified and written as mandatory, testable behavior.

10.1 Authentication & Role Management

Priority: High. Primary owner: Annamalai N.

Requirement ID	Requirement
FR-AUTH-001	The system shall allow a registered user to log in using valid credentials.
FR-AUTH-002	The system shall reject authentication when supplied credentials are invalid.
FR-AUTH-003	The system shall prevent a deactivated user from starting an authenticated session.
FR-AUTH-004	The system shall terminate the authenticated session when the user logs out.
FR-AUTH-005	The system shall associate each active user with an application role.
FR-AUTH-006	Only authorized administrators shall create, update, deactivate or reactivate user accounts.
FR-AUTH-007	The backend shall enforce role permissions for protected API operations.
FR-AUTH-008	The system shall return an access-denied response for operations outside the user's permission set.

10.2 Product & Category Management

Priority: High. Primary owner: Apurv Kumar Singh.

Requirement ID	Requirement
FR-PROD-001	The system shall allow authorized users to create a product with unique SKU, name, category, selling price and minimum stock level.
FR-PROD-002	The system shall reject creation/update of a product when its SKU duplicates an existing active SKU.
FR-PROD-003	The system shall allow authorized users to update product master data subject to validation.
FR-PROD-004	The system shall allow authorized users to create and maintain product categories.
FR-PROD-005	The system shall allow permitted users to search products by SKU or name.
FR-PROD-006	The system shall prevent physical deletion of products referenced by historical transactions; such products shall be deactivated.

10.3 Inventory Management

Priority: High. Primary owner: Apurv Kumar Singh.

Requirement ID	Requirement
FR-INV-001	The system shall display current available quantity for each active product.
FR-INV-002	The system shall allow permitted users to record a stock adjustment with product, quantity change, reason, user and timestamp.
FR-INV-003	The system shall reject an adjustment that would result in negative available quantity.
FR-INV-004	The system shall identify a product as low stock when available quantity is less than or equal to minimum stock.
FR-INV-005	The system shall record purchase, sale and adjustment events in inventory history.
FR-INV-006	The system shall provide inventory history filterable by product and date range.

10.4 Supplier & Purchase Management

Priority: High. Primary owner: Atharv Mittal.

Requirement ID	Requirement
FR-PUR-001	The system shall allow authorized users to create and update supplier records with required details.
FR-PUR-002	The system shall allow suppliers to be deactivated without deleting historical purchase records.
FR-PUR-003	The system shall allow purchase staff to create a purchase order for an active supplier.
FR-PUR-004	A purchase order shall contain at least one item with product, quantity and purchase price.
FR-PUR-005	The system shall calculate purchase order totals from item quantities and purchase prices.
FR-PUR-006	The system shall allow purchase staff to record received quantities against an existing purchase order.
FR-PUR-007	A successfully recorded receipt shall increase product inventory by the received quantity.
FR-PUR-008	The system shall prevent received quantity from exceeding ordered quantity unless an approved exception process exists.
FR-PUR-009	The system shall provide purchase history filterable by supplier and date range.

10.5 Sales Management

Priority: High. Primary owner: Avaneesh H D.

Requirement ID	Requirement
FR-SALE-001	The system shall allow authorized sales users to create a sale containing one or more items.
FR-SALE-002	The system shall validate each sale quantity as a positive integer.
FR-SALE-003	The system shall prevent a sale from being completed when requested quantity exceeds available stock.
FR-SALE-004	The system shall calculate the sale total from item quantities and selling prices.
FR-SALE-005	A completed sale shall decrease product inventory by the quantity sold.
FR-SALE-006	The system shall assign a unique transaction identifier to each completed sale.
FR-SALE-007	The system shall provide sales history filterable by transaction identifier and date range.

10.6 Reporting & Monitoring

Priority: High. Primary owner: Avaneesh H D.

Requirement ID	Requirement
FR-REP-001	The system shall provide an inventory report containing product, SKU, category, available quantity and stock status.
FR-REP-002	The system shall provide a low-stock report containing products at or below minimum stock.
FR-REP-003	The system shall provide a purchase report filterable by supplier and date range.
FR-REP-004	The system shall provide a sales report filterable by date range.
FR-REP-005	The system shall display report totals consistently with underlying stored transactions.

10.7 Common Validation & Search

Priority: Medium. Primary owner: Shared.

Requirement ID	Requirement
FR-COM-001	The system shall validate required fields before create/update operations.
FR-COM-002	The system shall display a clear validation message for each invalid input that prevents submission.
FR-COM-003	The system shall display a success/confirmation message after successful create, update, receipt, adjustment or sale operations.
FR-COM-004	The system shall display an explicit empty-state message when a valid search/filter returns no records.
FR-COM-005	The system shall preserve referential integrity when records are updated or deactivated.

11. Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	At least 95% of normal authenticated API requests shall respond within 2 seconds under the defined test load.	Performance test
NFR-002	Availability	The application shall achieve at least 99% availability during the defined operating window, excluding planned maintenance.	Operational review
NFR-003	Usability	A trained user shall complete a basic product search and stock lookup within 3 primary screens.	Usability walkthrough

ID	Category	Measurable Requirement	Verification
NFR-004	Reliability	Retrying the same failed transaction request shall not create duplicate purchase or sale records.	Integration/negative test
NFR-005	Maintainability	Business functionality shall be separated into identifiable modules with automated tests for core services.	Code review
NFR-006	Testability	Core API services shall expose deterministic behavior suitable for automated unit/integration tests.	Test execution
NFR-007	Portability	The web client shall work on current stable Chrome/Chromium, Firefox and Edge during final testing.	Cross-browser test
NFR-008	Scalability	The schema shall support at least 10,000 products and 100,000 transaction/item records without schema redesign.	Data-volume test
NFR-009	Data Integrity	Purchase, sale and inventory updates shall be atomic so partial updates cannot leave inconsistent stock.	Database/integration test
NFR-010	Usability	Validation/error messages shall identify the field or operation needing correction.	UI test
NFR-011	Maintainability	Code shall follow agreed formatting, naming, modularity, Git review and static-analysis practices.	Code review/SonarQube
NFR-012	Auditability	Inventory-affecting events shall retain responsible user and timestamp where applicable.	Database inspection/test

12. Security Requirements

ID	Security Requirement	Verification
SEC-001	The system shall authenticate users before granting access to protected functions.	Authentication test
SEC-002	Passwords shall not be stored in plaintext and shall use secure password hashing.	Code/database review
SEC-003	Authorization shall be enforced at the server/API layer.	Authorization test
SEC-004	Client-side manipulation shall not allow access outside the user's permissions.	Negative API test
SEC-005	Credentials and authenticated session/token data shall be transmitted only over HTTPS in production.	Network/deployment inspection
SEC-006	The backend shall validate and sanitize client-supplied input before database operations.	Security tests
SEC-007	Secrets and database credentials shall not be exposed in client-side code.	Configuration/code review
SEC-008	Authentication failures shall not reveal whether a particular account exists.	Security test
SEC-009	Inactive accounts shall not authenticate successfully.	Authentication test
SEC-010	Database access shall use least-privilege application credentials.	Deployment review

13. Business / Domain Rules

ID	Business / Domain Rule
BR-001	Every active product shall have a unique SKU.
BR-002	Available stock quantity shall never be negative.
BR-003	Low stock means available quantity is less than or equal to minimum stock.
BR-004	Only active suppliers may be used for new purchase orders.
BR-005	A valid purchase order shall contain at least one item.
BR-006	Receiving a purchase increases inventory by the quantity actually received.
BR-007	A completed sale decreases inventory by the quantity sold.
BR-008	A sale cannot be completed when requested quantity exceeds available stock.
BR-009	Historical transactions remain valid when a product or supplier is deactivated.
BR-010	Only authorized roles may perform administrative, inventory-adjustment, purchasing or sales operations.
BR-011	Transaction totals shall be derived from stored line quantities and unit prices.
BR-012	Inventory changes shall be traceable to their source purchase, sale or adjustment event.

13.1 Permission Matrix

Function	Admin	Inventory	Purchase	Sales	Manager
User/Role Management	C/U/D	-	-	-	View
Product/Category	C/U/D*	C/U	View	View	View
Inventory View	Yes	Yes	Yes	Yes	Yes
Inventory Adjustment	Yes	Yes	-	-	Policy
Supplier Management	C/U/D*	View	C/U/D*	View	View
Purchase Orders	Yes	View	C/U/Receive	View	View
Sales	Yes	View	View	Create/View	View
Reports	All	Inventory	Purchase	Sales	All

* C = Create, U = Update, D = Deactivate/retire. Historical records shall not be physically deleted when references exist.

14. Database / Field Requirements

These are the initial logical field requirements. Physical column names, indexes and implementation-specific constraints may be refined during database design without changing the intended behavior.

14.1 User and Role Fields

Entity	Field	Type	Length/Range	Mandatory	Description
User	user_id	PK	Generated	Y	Unique user identifier
User	name	String	1–100	Y	Display/full name
User	email	String	Valid email	Y	Unique login/contact identifier
User	password_hash	String	Implementation-defined	Y	Secure password hash
User	role_id	FK	Valid role	Y	Assigned role

Entity	Field	Type	Length/Range	Mandatory	Description
User	status	Enum	ACTIVE/INACTIVE	Y	Account status
Role	role_id	PK	Generated	Y	Unique role ID
Role	role_name	String	1-50	Y	Role name

14.2 Product, Category and Inventory Fields

Entity	Field	Type	Length/Range	Mandatory	Description
Category	category_id	PK	Generated	Y	Unique category ID
Category	name	String	1-80	Y	Category name
Product	product_id	PK	Generated	Y	Unique product ID
Product	sku	String	1-40	Y	Unique stock keeping unit
Product	name	String	1-150	Y	Product name
Product	category_id	FK	Valid category	Y	Category reference
Product	selling_price	Decimal	>=0	Y	Unit selling price
Product	minimum_stock	Integer	>=0	Y	Low-stock threshold
Product	status	Enum	ACTIVE/INACTIVE	Y	Product status
Inventory	product_id	FK/PK	Valid product	Y	Referenced product
Inventory	available_qty	Integer	>=0	Y	Current stock
InventoryTransaction	transaction_id	PK	Generated	Y	Inventory event ID
InventoryTransaction	product_id	FK	Valid product	Y	Affected product
InventoryTransaction	change_qty	Integer	Signed	Y	Quantity change
InventoryTransaction	source_type	Enum	PURCHASE/SALE/A DJUSTMENT	Y	Origin
InventoryTransaction	source_id	Reference	Valid source	Y	Source transaction ID
InventoryTransaction	created_by	FK	Valid user	Y	Responsible user
InventoryTransaction	created_at	Timestamp	Generated	Y	Event timestamp

14.3 Supplier and Purchase Fields

Entity	Field	Type	Length/Range	Mandatory	Description
Supplier	supplier_id	PK	Generated	Y	Supplier ID
Supplier	name	String	1-150	Y	Supplier/business name
Supplier	contact_name	String	0-100	N	Primary contact
Supplier	phone	String	Valid phone	N	Contact number
Supplier	email	String	Valid email	N	Contact email
Supplier	address	String	0-250	N	Supplier address
Supplier	status	Enum	ACTIVE/INACTIVE	Y	Supplier status
PurchaseOrder	purchase_id	PK	Generated	Y	Purchase order ID
PurchaseOrder	supplier_id	FK	Valid supplier	Y	Supplier reference
PurchaseOrder	order_date	Date	Valid date	Y	Order date
PurchaseOrder	status	Enum	DRAFT/ORDERED/P ARTIAL/RECEIVED/ CANCELLED	Y	Purchase status
PurchaseOrder	total_amount	Decimal	>=0	Y	Derived order total
PurchaseItem	purchase_item_id	PK	Generated	Y	Line ID
PurchaseItem	purchase_id	FK	Valid purchase	Y	Parent purchase
PurchaseItem	product_id	FK	Valid product	Y	Product reference
PurchaseItem	ordered_qty	Integer	>0	Y	Ordered quantity
PurchaseItem	received_qty	Integer	0..ordered	Y	Received quantity
PurchaseItem	unit_cost	Decimal	>=0	Y	Purchase unit cost

14.4 Sales Fields

Entity	Field	Type	Length/Range	Mandatory	Description
Sale	sale_id	PK	Generated	Y	Sale identifier
Sale	transaction_no	String	Unique	Y	Human-readable transaction reference
Sale	sale_date	Timestamp	Generated	Y	Sale timestamp
Sale	created_by	FK	Valid user	Y	Sales user
Sale	total_amount	Decimal	>=0	Y	Derived sale total
SaleItem	sale_item_id	PK	Generated	Y	Line ID
SaleItem	sale_id	FK	Valid sale	Y	Parent sale
SaleItem	product_id	FK	Valid product	Y	Sold product

Entity	Field	Type	Length/Range	Mandatory	Description
SaleItem	quantity	Integer	>0	Y	Quantity sold
SaleItem	unit_price	Decimal	>=0	Y	Price at sale time
SaleItem	line_total	Decimal	>=0	Y	Quantity × unit price

14.5 Report Requirements

Report	Required Fields / Filters
Inventory Report	SKU, product, category, available quantity, minimum stock, stock status; category/status filters.
Low-Stock Report	SKU, product, available quantity, minimum stock, status; category filter.
Purchase Report	Purchase ID, supplier, order date, status, total; supplier/date filters.
Sales Report	Transaction number, sale date, total, created by; date filter.
Inventory History	Timestamp, product, change quantity, source type/source ID, user; product/date filters.

15. Glossary

Term	Definition
API	Application Programming Interface used for communication between software components.
CRUD	Create, Read, Update and Delete operations on data.
Django REST Framework	Python framework planned for implementing REST APIs.
Inventory	Current quantity of products available in the store.
Inventory Adjustment	Authorized stock quantity change recorded with a reason.
Low Stock	Condition where available quantity is less than or equal to minimum stock.
Purchase Order	Record representing goods ordered from a supplier.
Purchase Receipt	Record of goods actually received against a purchase order.
RBAC	Role-Based Access Control; permissions are assigned according to user roles.
REST	Representational State Transfer style used for web APIs.
SKU	Stock Keeping Unit; unique product identifier.
PostgreSQL	Relational database planned for persistent application data.
React	JavaScript library planned for the web frontend.
Sale	Recorded transaction in which one or more products are sold.
Supplier	Business/entity from which the store procures products.
TechZone Electronics	Fictional electronics retail shop used as the project case-study context.

16. Requirement Traceability Matrix

The RTM links selected requirements to planned architecture, design/use-case, code and test artifacts. Since implementation is not yet complete, code and test references are planned identifiers and shall be updated during development.

Req ID	Brief Description	Architecture	Design/UC	Planned Code	Test ID
FR-AUTH-001	Valid login	AUTH-MOD	UC-01	auth/	TC-AUTH-01
FR-AUTH-007	Backend authorization	AUTH-SEC	UC-02	permissions/	TC-SEC-03
FR-PROD-001	Create product	PRODUCT-MOD	UC-03	products/	TC-PROD-01
FR-PROD-002	Reject duplicate SKU	PRODUCT-MOD	UC-03	products/validators	TC-PROD-02
FR-INV-001	Display stock	INVENTORY-MOD	UC-04	inventory/	TC-INV-01
FR-INV-004	Low-stock identification	INVENTORY-MOD	UC-04/09	inventory/rules	TC-INV-04
FR-PUR-003	Create purchase order	PURCHASE-MOD	UC-06	purchases/	TC-PUR-03
FR-PUR-007	Increase stock on receipt	PURCHASE+INVENTORY	UC-07	purchases/receipt	TC-PUR-07
FR-SALE-003	Prevent sale above stock	SALES+INVENTORY	UC-08	sales/	TC-SALE-03

Req ID	Brief Description	Architecture	Design/UC	Planned Code	Test ID
FR-SALE-005	Decrease stock after sale	SALES+INVENTORY	UC-08	sales/checkout	TC-SALE-05
FR-REP-001	Inventory report	REPORT-MOD	UC-09	reports/	TC-REP-01
FR-REP-004	Sales report	REPORT-MOD	UC-09	reports/	TC-REP-04
FR-COM-001	Required-field validation	COMMON-MOD	All relevant	validators/	TC-COM-01
NFR-001	2-second API target	ARCH-PERF	System	API/services	TC-NFR-01
NFR-009	Atomic updates	DB-ARCH	UC-07/08	transaction services	TC-NFR-09
SEC-002	Password hashing	AUTH-SEC	UC-01	auth config	TC-SEC-02
SEC-004	Block unauthorized API access	AUTH-SEC	UC-02+	permissions/	TC-SEC-04
BR-003	Low-stock rule	INVENTORY-DOMAIN	UC-04/09	inventory rules	TC-BR-03
BR-008	Sale cannot exceed stock	SALES-DOMAIN	UC-08	sales rules	TC-BR-08

16.1 Requirement Ownership Coverage

Team Member	Primary Functional Area	Requirement Groups Owned
Annamalai N	Authentication & Role Management	FR-AUTH-001 to FR-AUTH-008; related SEC requirements
Apurv Kumar Singh	Product & Inventory Management	FR-PROD-001 to FR-PROD-006; FR-INV-001 to FR-INV-006
Atharv Mittal	Supplier & Purchase Management	FR-PUR-001 to FR-PUR-009
Avaneesh H D	Sales & Reporting Management	FR-SALE-001 to FR-SALE-007; FR-REP-001 to FR-REP-005

Shared requirements cover validation, common UI behavior, data integrity, testing, documentation and cross-module integration. Each member therefore owns a complete business functionality.

Conclusion

This SRS establishes the initial requirements baseline for the TechZone Electronics Inventory Management System. It defines scope, users, functions, interfaces, use cases, measurable quality requirements, security controls, business rules, logical data fields, glossary terms and traceability. Approved changes shall be reflected through the revision history and RTM.

PART II — SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION (SAD)

This specification defines the proposed architecture and design of the Electronics Store Inventory Management System. It is based on the SRS and synopsis supplied by Team 9. Where the SRS describes a planned technology or integration, this document treats it as a design decision rather than as evidence that implementation has already been completed.

1. SAD Purpose and Scope

The purpose of this SAD is to translate the approved requirements baseline into a coherent architecture and design that can guide implementation, integration, testing and review. The design covers the web client, REST API, business modules, persistence layer, security controls, transaction workflows, logical data relationships and deployment environment.

- Define the overall client-server architecture for React, Django REST Framework and PostgreSQL.
- Separate business functionality into identifiable modules aligned with the team's ownership areas.
- Define how authentication, authorization, validation and inventory-changing transactions are handled.
- Provide logical database and component views that support the SRS entities and requirements.
- Provide design-level traceability from important SRS requirements to architecture components and test identifiers.

2. Design Drivers from the SRS

Driver	Design Implication
Client-server web architecture	React browser client communicates with Django REST APIs; the database is not exposed directly to browsers.
Role-based authorization	Authorization is enforced at the backend/API layer, not only by hiding frontend controls.
Inventory consistency	Purchase receipt and sale operations update inventory atomically with their source transaction.
Traceability	Inventory-affecting events retain source information, responsible user and timestamp where applicable.
Security	Passwords are securely hashed; production credentials/session data use HTTPS; input is validated server-side.
Modularity	Authentication, product/inventory, supplier/purchase and sales/reporting are separated into functional modules.
Testability	Core API/business services expose deterministic behavior suitable for unit and integration testing.

3. Architectural Overview

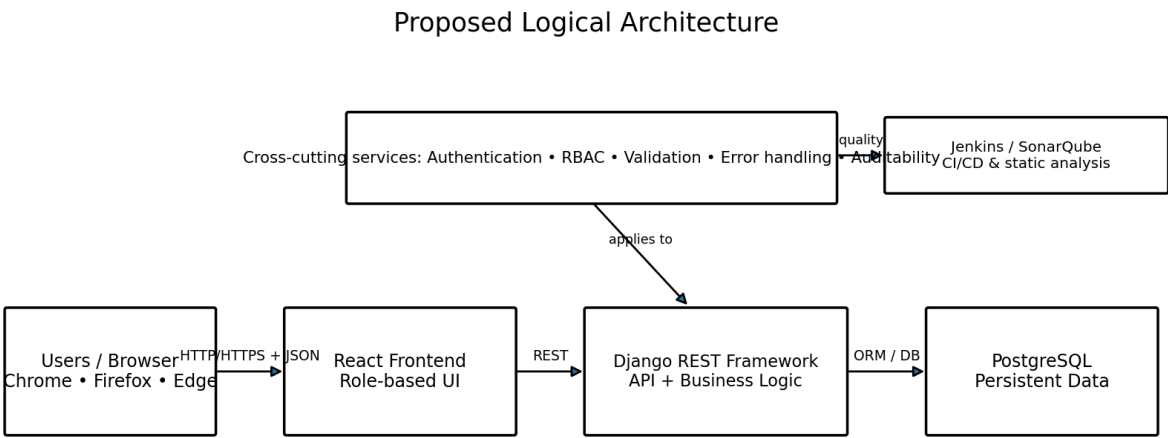


Figure SAD-01. Proposed logical architecture derived from the SRS technology stack and interfaces.

The architecture uses a browser-based React client, a Django REST Framework backend and PostgreSQL persistence. The backend is the enforcement point for business rules and role authorization. Jenkins and SonarQube are treated as development/quality infrastructure identified in the project synopsis and SRS.

4. System Context

The system is an internal application for authorized TechZone Electronics staff. The primary human roles identified by the SRS are Administrator, Inventory Staff, Purchase Staff, Sales Staff and Manager/Owner. Suppliers are an external business stakeholder whose details are maintained through the purchasing workflow.

Actor / External Element	Interaction with System
Administrator	Manages users/roles and broad master data; reviews reports.
Inventory Staff	Maintains products and stock, performs permitted adjustments and monitors low stock.
Purchase Staff	Maintains suppliers, creates purchase orders and records received goods.
Sales Staff	Records sales after product/stock validation.
Manager / Owner	Reviews inventory, purchase, sales and low-stock reports.
Supplier	Provides goods represented by supplier and purchase records.
Jenkins / SonarQube	Supports automated build/test pipeline and static-analysis checks.

5. Use Case Design View

Use Case View (UC-01 to UC-10)

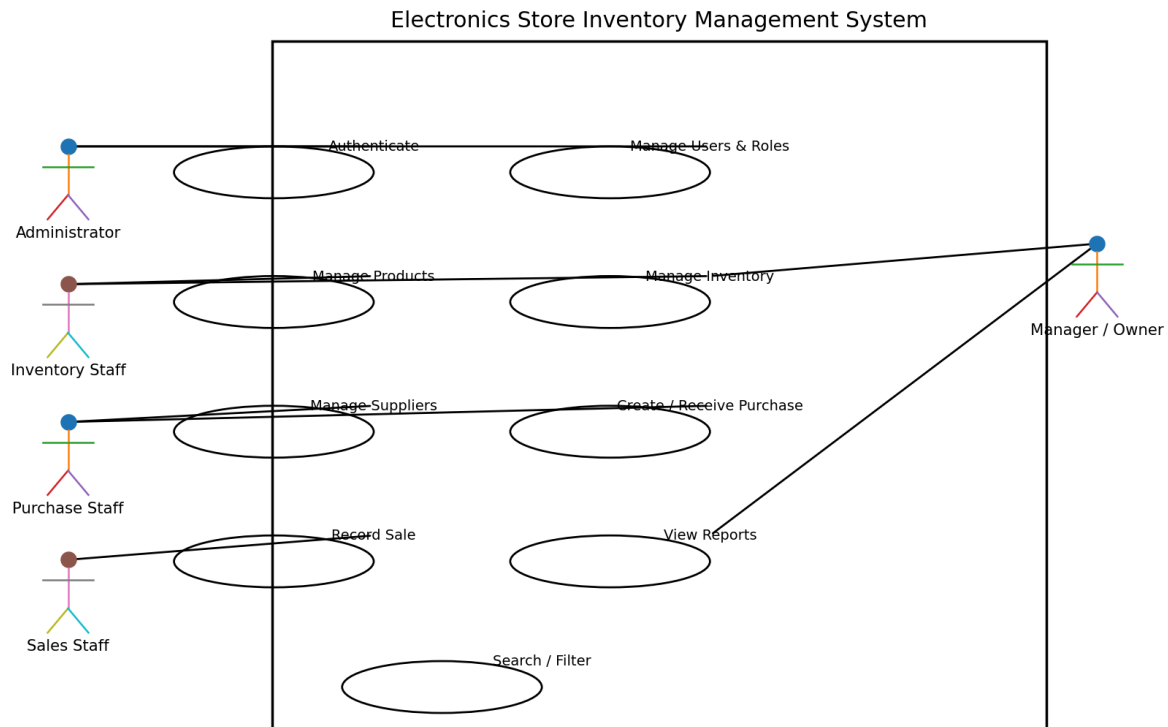


Figure SAD-02. Use case view based on UC-01 through UC-10 in the SRS.

The use cases form the primary behavioral boundary of the system: authenticate, manage users/roles, manage products, manage inventory, manage suppliers, create/receive purchases, record sales, view reports and search/filter records.

6. Layered / Module Architecture

Application Component / Module Decomposition

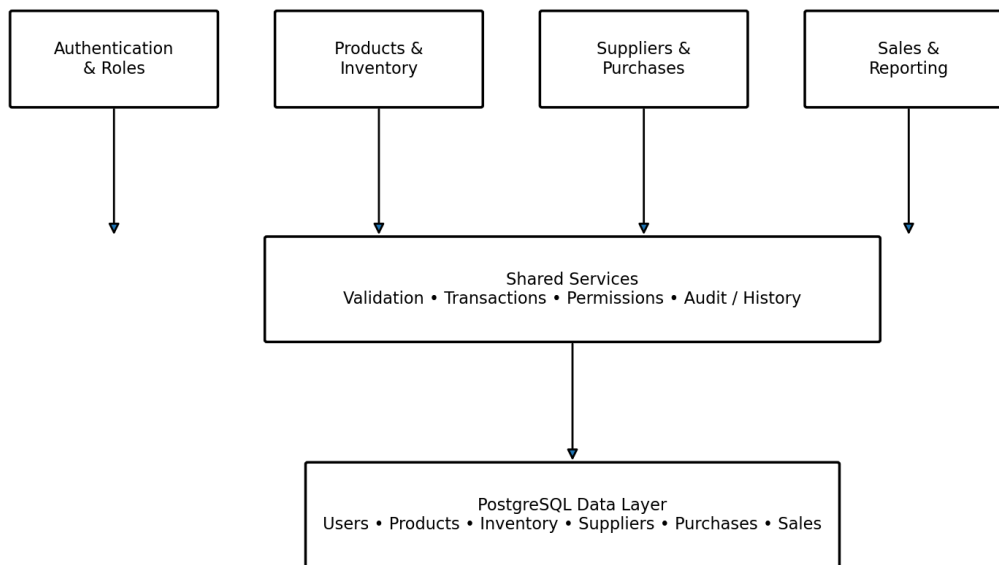


Figure SAD-03. Functional module decomposition.

The four primary business areas follow the functional ownership defined in the synopsis and SRS. Shared services provide validation, transaction handling, permission enforcement and audit/history support. This separation allows each team member to implement a coherent feature area while preserving cross-module integration.

7. Module Responsibilities

Module	Main Responsibilities	Key SRS Coverage
Authentication & Roles	Login/logout, account state, role assignment, API authorization.	F-01; UC-01/02; FR-AUTH-*; SEC-*
Products & Inventory	Categories, products, SKU uniqueness, stock quantity, adjustments, low-stock logic, history.	F-02/F-03; UC-03/04; FR-PROD-*; FR-INV-*
Suppliers & Purchases	Supplier records, purchase orders, receipt recording and stock increase.	F-04/F-05; UC-05/06/07; FR-PUR-*
Sales & Reporting	Sales transactions, stock decrease, transaction history, reports and monitoring.	F-06/F-07; UC-08/09; FR-SALE-*; FR-REP-*
Common Services	Required-field validation, search/filtering, messages, referential integrity and shared error handling.	F-08; FR-COM-*

8. Data Design

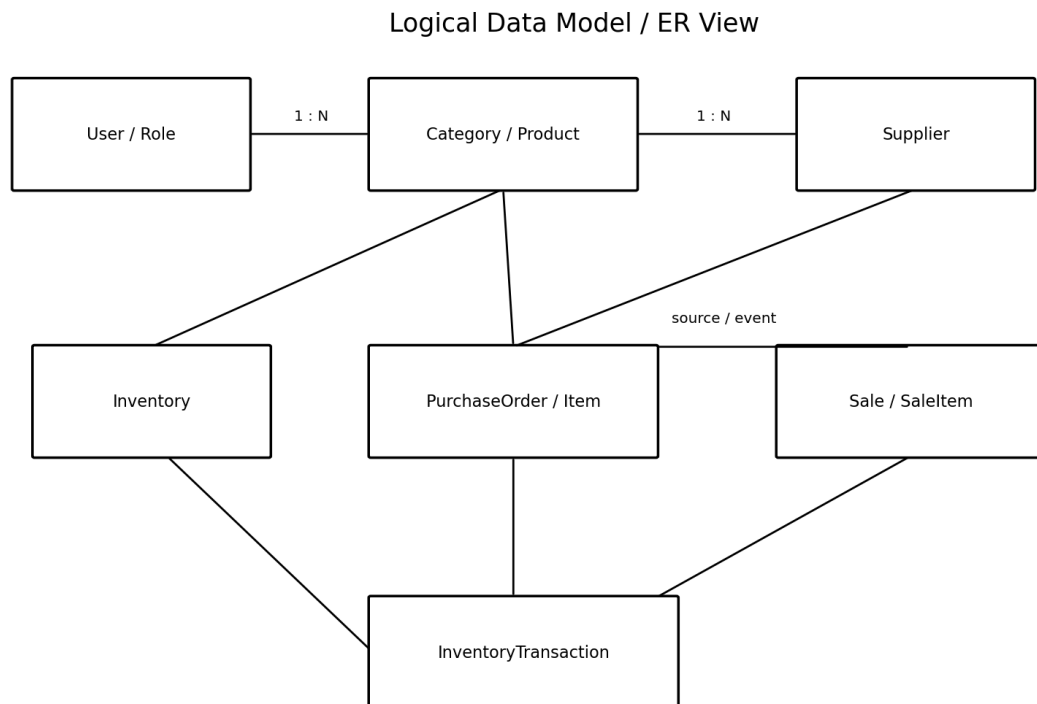


Figure SAD-04. Logical entity relationship view based on the SRS database/field requirements.

The logical model contains the entities identified by the SRS: User, Role, Category, Product, Inventory, InventoryTransaction, Supplier, PurchaseOrder, PurchaseItem, Sale and SaleItem. The design preserves historical transaction references when products or suppliers are deactivated rather than physically deleting referenced records.

9. Key Data Integrity Rules

- Each active product has a unique SKU (BR-001).
- Available stock never becomes negative (BR-002).
- Low stock is defined as available quantity less than or equal to minimum stock (BR-003).
- Only active suppliers can be used for new purchase orders (BR-004).
- A purchase order contains at least one item (BR-005).
- Receiving a purchase increases inventory by the quantity actually received (BR-006).
- A completed sale decreases inventory by the quantity sold (BR-007).
- A sale is rejected when requested quantity exceeds available stock (BR-008).
- Inventory changes are traceable to purchase, sale or adjustment events (BR-012).

10. Interface / API Design

The SRS specifies JSON-based communication between React and the Django REST API, with PostgreSQL accessed through the backend/database layer. The exact URL naming convention was not specified in the supplied documents; therefore the following is a design convention that can be adopted during implementation.

Resource Area	Representative Operations	Validation / Security
Authentication	Login, logout, session/token handling	Credentials validated; inactive users rejected.
Users / Roles	Create, update, deactivate/reactivate users	Administrator-only; role permissions enforced server-side.

Products / Categories	Create, update, deactivate, search	Unique SKU; required fields; historical references preserved.
Inventory	View, adjust, history, low-stock	Non-negative quantity; adjustment reason/user/timestamp.
Suppliers	Create, update, deactivate, search	Only active suppliers eligible for new purchase orders.
Purchases	Create order, receive goods, purchase history	At least one item; received quantity bounded by order; atomic stock update.
Sales	Create sale, sale history	Positive quantities; insufficient stock rejected; atomic stock update.
Reports	Inventory, low-stock, purchase, sales, history	Authorized access; filters validated.

11. Purchase Receipt Sequence Design

Sequence Design — Receive Purchase (UC-07)

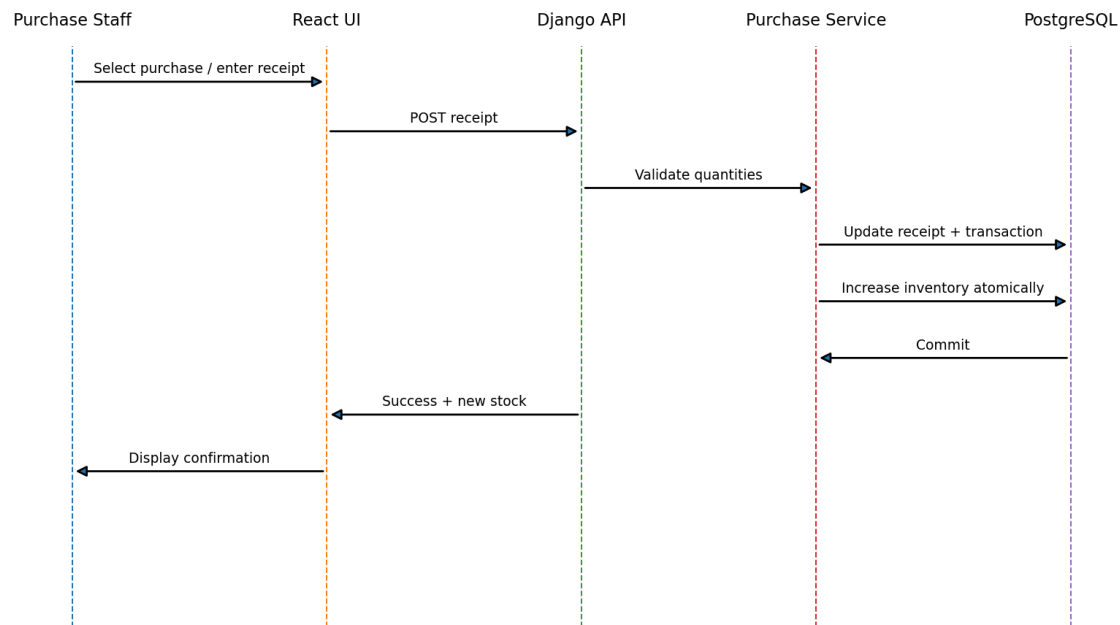


Figure SAD-05. Sequence design for UC-07 Receive Purchase.

The intended transaction boundary is important: recording the receipt and increasing the corresponding inventory should succeed together. The SRS NFR-009 explicitly requires atomic purchase, sale and inventory updates so that partial updates do not leave inconsistent stock.

12. Sale Sequence Design

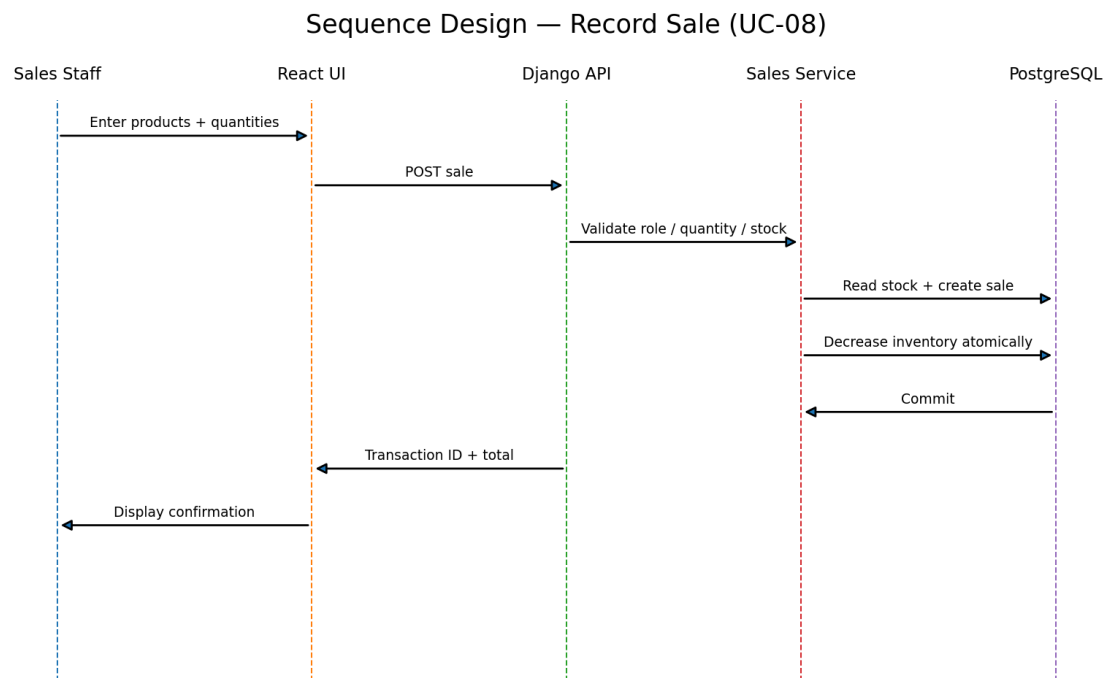


Figure SAD-06. Sequence design for UC-08 Record Sale.

Before a sale is committed, the API validates authorization, positive quantities and available stock. On successful completion, the sale receives a unique transaction identifier and inventory is decreased by the quantity sold.

13. Deployment Design

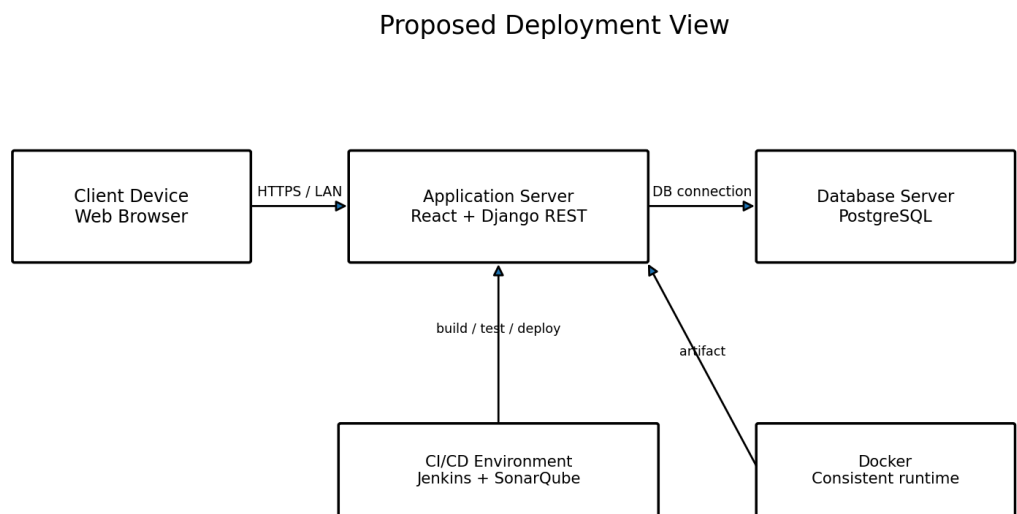


Figure SAD-07. Proposed deployment view.

The SRS identifies a Linux-based server environment, browser clients, HTTPS for production communication, Docker for consistent environments, Jenkins for CI/CD and SonarQube or similar tooling for static analysis. Exact hosting topology was not specified, so this view remains a deployment-level proposal.

14. UI / Navigation Design

Screen / View	Primary Users	Core Elements
Login	All users	Credentials, validation, authentication failure message.
Dashboard	All authenticated users	Role-appropriate navigation and operational summary.
Products / Categories	Admin, Inventory	Tables/forms, SKU, category, price, minimum stock, status, search.
Inventory	Admin, Inventory, permitted roles	Available quantity, low-stock status, adjustments, history.
Suppliers / Purchases	Admin, Purchase	Supplier selection, order items, quantities, totals, status, receipt action.
Sales	Sales, permitted roles	Product search, quantity entry, stock validation, totals, confirmation.
Reports	Manager, Admin, relevant roles	Inventory, low-stock, purchase, sales and history filters.

15. Security Architecture

- Authentication precedes protected operations (SEC-001).
- Passwords are stored only as secure hashes, never plaintext (SEC-002).
- Authorization is enforced at the API/server layer (SEC-003).
- Manually modified client requests cannot bypass permissions (SEC-004).
- Production credentials and authenticated session/token data use HTTPS (SEC-005).
- Backend input is validated/sanitized before database operations (SEC-006).
- Secrets and database credentials are kept out of client-side code (SEC-007).
- Authentication failures avoid revealing whether an account exists (SEC-008).
- Inactive accounts cannot authenticate (SEC-009).
- Database access uses least-privilege application credentials (SEC-010).

16. Error Handling and Validation Design

Condition	Design Response
Invalid credentials	Reject authentication and show a generic failure message.
Inactive account	Reject authentication; do not create an authenticated session.
Duplicate SKU	Reject product create/update and identify the conflicting field.
Invalid quantity / negative stock	Reject transaction before persistence.
Inactive supplier	Reject creation of a new purchase order.
Over-receipt	Reject receipt when received quantity exceeds the allowed quantity.
Insufficient sale stock	Reject the sale and do not create a completed sale.
Invalid report filter	Display validation feedback and do not return misleading results.
No search results	Display an explicit empty-state message.
Unauthorized API operation	Return an access-denied response from the backend.

17. NFR Design Considerations

NFR	Design Consideration
NFR-001 Performance	Use efficient queries, indexed lookup fields and avoid unnecessary API/database round trips; validate the 2-second target under defined test load.
NFR-002 Availability	Use the planned deployment infrastructure and monitor planned maintenance separately.
NFR-004 Reliability	Use transaction identifiers/idempotency or equivalent duplicate-protection behavior for retried operations.
NFR-007 Portability	Perform final UI testing on current stable Chrome/Chromium, Firefox and Edge.

NFR-008 Scalability	Use relational constraints/indexing appropriate for at least 10,000 products and 100,000 transaction/item records.
NFR-009 Data Integrity	Wrap purchase receipt, sale and inventory changes in atomic database transactions.
NFR-012 Auditability	Persist responsible user and timestamps for inventory-affecting events.

18. Design Traceability

SRS Requirement	Design Element	Planned Test
FR-AUTH-001	Authentication module / login flow	TC-AUTH-01
FR-AUTH-007	API authorization middleware/permissions	TC-SEC-03
FR-PROD-001	Product service + SKU constraint	TC-PROD-01
FR-INV-004	Inventory low-stock rule	TC-INV-04
FR-PUR-003	Purchase module / order workflow	TC-PUR-03
FR-PUR-007	Purchase receipt + inventory transaction	TC-PUR-07
FR-SALE-003	Sales service + stock validation	TC-SALE-03
FR-SALE-005	Sale + inventory atomic transaction	TC-SALE-05
FR-REP-001	Reporting module	TC-REP-01
FR-COM-001	Shared validation layer	TC-COM-01
NFR-001	Performance design	TC-NFR-01
NFR-009	Database transaction boundary	TC-NFR-09
SEC-002	Authentication/password configuration	TC-SEC-02
SEC-004	Server-side authorization	TC-SEC-04
BR-003	Inventory low-stock domain rule	TC-BR-03
BR-008	Sales stock validation rule	TC-BR-08

19. Design Assumptions and Open Decisions

- The supplied SRS defines the technology stack and logical data requirements, but it does not specify final physical database indexes, exact REST endpoint paths, hosting provider, token implementation details or final screen mock-ups.
- Those implementation-specific decisions should be recorded in Git/Jira and reflected in later revisions if they change externally visible behavior.
- The architecture shown here is therefore the proposed design baseline for implementation and testing, not a claim that every component has already been deployed.

20. SAD Conclusion

The proposed design maps the SRS into a modular client-server architecture with clear separation between presentation, API/business logic and persistence. The design explicitly protects the inventory invariants that are central to the project: purchases increase stock only when valid receipts are recorded, sales cannot exceed available stock, and inventory-affecting events remain traceable. The architecture also provides explicit locations for authentication, RBAC, validation, reporting and quality tooling.

PART III — SOFTWARE TEST PLAN (STP)

This Software Test Plan defines how Team 9 will validate the Electronics Store Inventory Management System against the supplied SRS. Because the supplied materials describe a planned implementation and do not contain execution results, the test cases below are planned test cases with expected results; they are not presented as completed execution evidence.

1. Purpose

The STP establishes the testing scope, strategy, environment, responsibilities, entry/exit criteria, test cases, security testing, non-functional testing and requirements traceability for the project.

2. Test Scope

2.1 Features to be Tested

- Authentication and role-based access control.
- Product and category management.
- Inventory quantities, adjustments, low-stock detection and history.
- Supplier and purchase-order management.
- Purchase receipt processing and inventory increase.
- Sales transactions and inventory decrease.
- Reports and search/filtering.
- Validation, error handling, data integrity and auditability.
- Security controls and relevant non-functional requirements.

2.2 Features Outside Test Scope

- Customer-facing e-commerce and online ordering.
- Online payment gateway integration.
- Delivery/logistics/shipment tracking.
- Full accounting, taxation, payroll or ERP behavior.
- AI demand forecasting.
- Enterprise multi-store management.
- Supplier payment settlement or banking integration.

3. Test Objectives

5. Verify that each functional requirement behaves as specified.
6. Verify that business rules preserve inventory correctness.
7. Verify that role permissions are enforced by the backend/API.
8. Verify that invalid and maliciously manipulated inputs are rejected safely.
9. Verify the SRS non-functional targets that can be measured in the project environment.
10. Provide traceability from requirements to planned test cases.
11. Identify defects early and prevent regression of integrated functionality.

4. Test Strategy

Test Level	Purpose	Typical Technique
Unit Testing	Validate isolated services, validators, rules and calculations.	pytest/unit tests
API / Integration Testing	Validate API contracts, authorization, database interactions and module integration.	API tests + integration fixtures
System Testing	Validate end-to-end business workflows in the browser/API.	Functional system tests
Security Testing	Validate authentication, authorization, input handling and secrets/configuration controls.	Negative API tests, review and configuration inspection
Performance Testing	Validate the SRS response-time and data-volume targets under a defined load.	Load/performance test

Compatibility Testing	Validate browser portability.	Chrome/Chromium, Firefox, Edge
Regression Testing	Ensure fixes do not break previously passing functionality.	Automated regression suite
Acceptance / Demo Validation	Validate that the completed prototype supports the intended internal store workflows.	Scenario-based walkthrough

5. Test Levels and Coverage

The test pyramid is planned from deterministic unit tests at the business-rule layer through API/integration tests and finally system-level workflows. Security and non-functional tests cut across these levels.

- Unit: validators, totals, stock rules, low-stock logic, permission decisions.
- Integration: purchase receipt + inventory update, sale + inventory update, database constraints, authorization.
- System: login → setup → purchase → receipt → inventory → sale → reporting.
- Security: unauthorized API calls, input manipulation, authentication edge cases and configuration review.
- NFR: response time, duplicate-request behavior, atomicity, browser compatibility, data volume and auditability.

6. Test Environment

Area	Planned Environment
Client	Desktop/laptop browser.
Browsers	Current stable Chrome/Chromium, Firefox and Edge.
Frontend	React with JavaScript.
Backend	Python + Django REST Framework.
Database	PostgreSQL.
Test Framework	pytest plus API/integration/system testing tools.
CI/CD / Quality	Jenkins and SonarQube or similar, as applicable.
Runtime	Linux-based development/deployment environment; Docker may be used for consistency.
Network	LAN/Wi-Fi during development; HTTPS in production-like testing.

7. Test Data Strategy

- Create users for Administrator, Inventory Staff, Purchase Staff, Sales Staff and Manager/Owner roles.
- Create active and inactive users to validate authentication and authorization.
- Create products with unique and duplicate SKUs, different minimum-stock thresholds and different stock quantities.
- Create active and inactive suppliers and purchase orders in different statuses.
- Create purchase orders with one or more items, partial receipts and invalid receipt quantities.
- Create sales data with sufficient and insufficient stock conditions.
- Create historical inventory events for report and filter validation.
- Use synthetic credentials and data; do not use real customer or production secrets in test data.

8. Entry and Exit Criteria

Entry Criteria	Exit Criteria
Approved SRS/SAD baseline available.	All planned critical/high-priority functional tests executed.
Test environment and database available.	No open blocker/critical defect; high-severity defects dispositioned.
Required test data prepared.	Core purchase/sale/inventory integrity tests pass.
Build deployable to test environment.	Security tests executed and findings recorded.
Test cases reviewed by team.	Requirements traceability updated with execution results.

9. Roles and Responsibilities

Role	Responsibility
All Team Members	Prepare tests, review requirements, execute assigned tests, record evidence and review defects.
Annamalai N	Primary responsibility for authentication/RBAC test coverage.
Apurv Kumar Singh	Primary responsibility for product/inventory test coverage.

Atharv Mittal	Primary responsibility for supplier/purchase test coverage and receipt-to-inventory integration.
Avaneesh H D	Primary responsibility for sales/reporting test coverage.
Team Lead / Review	Coordinate integration, traceability, regression and final test-plan completeness.

10. Defect Management

- Each defect should include a unique identifier, requirement/test reference, description, environment, reproduction steps, expected result, actual result, severity, priority, owner and evidence.
- Severity should distinguish blocking/critical failures from major functional defects, minor defects and cosmetic issues.
- Fixed defects must be re-tested; impacted related cases should enter the regression set.
- The final submission should retain the test execution record and defect disposition in the team repository/Jira where applicable.

11. Functional Test Cases

Test ID	Requirement	Scenario	Test Procedure	Expected Result	Status
TC-AUTH-01	FR-AUTH-001	Valid login	Enter valid active-user credentials.	User is authenticated and role-appropriate dashboard is shown.	Planned
TC-AUTH-02	FR-AUTH-002	Invalid credentials	Enter incorrect password.	Access is denied; no authenticated session is created.	Planned
TC-AUTH-03	FR-AUTH-003 / SEC-009	Inactive user	Attempt login with inactive account.	Authentication is rejected.	Planned
TC-AUTH-04	FR-AUTH-004	Logout	Log in, then select logout.	Authenticated session is terminated.	Planned
TC-AUTH-05	FR-AUTH-005	Role assignment	Create/modify user with valid role.	User is associated with selected application role.	Planned
TC-SEC-03	FR-AUTH-007 / SEC-003	Backend authorization	Call protected API with insufficient role.	API rejects operation with access-denied response.	Planned
TC-PROD-01	FR-PROD-001	Create product	Enter valid SKU/name/category/price/minimum stock.	Product is created with unique SKU and supplied values.	Planned
TC-PROD-02	FR-PROD-002	Duplicate SKU	Create/update product using existing active SKU.	Operation is rejected; duplicate SKU is reported.	Planned
TC-PROD-03	FR-PROD-003	Update product	Edit valid product master data.	Changes are validated and persisted.	Planned
TC-PROD-04	FR-PROD-004	Category management	Create/update category with valid data.	Category is persisted and usable by products.	Planned
TC-PROD-05	FR-PROD-005	Product search	Search by SKU and by product name.	Matching products are returned.	Planned
TC-PROD-06	FR-PROD-006 / BR-009	Deactivate historical product	Deactivate product referenced by transaction.	Product becomes inactive; historical transaction remains valid.	Planned
TC-INV-01	FR-INV-001	Display stock	Open inventory for active products.	Current available quantity is displayed.	Planned
TC-INV-02	FR-INV-002	Stock adjustment	Submit permitted adjustment with reason.	Adjustment is saved with product, quantity, reason, user and timestamp.	Planned
TC-INV-03	FR-INV-003 / BR-002	Reject negative stock	Attempt adjustment causing negative quantity.	Adjustment is rejected and stock remains unchanged.	Planned
TC-INV-04	FR-INV-004 / BR-003	Low-stock identification	Set quantity equal to/below minimum stock.	Product is identified as low stock.	Planned
TC-INV-05	FR-INV-005	Inventory history	Perform purchase/sale/adjustment and inspect history.	Corresponding inventory events are recorded.	Planned
TC-INV-06	FR-INV-006	History filter	Filter inventory history by product/date.	Only matching events are shown.	Planned
TC-PUR-01	FR-PUR-001	Create supplier	Enter valid required supplier details.	Supplier is created.	Planned
TC-PUR-02	FR-PUR-002 / BR-009	Deactivate supplier	Deactivate supplier with purchase history.	Supplier becomes inactive; historical records remain.	Planned

TC-PUR-03	FR-PUR-003 / BR-004	Create purchase order	Select active supplier and add valid item.	Purchase order is created.	Planned
TC-PUR-04	FR-PUR-004 / BR-005	Purchase item required	Try to save purchase order without items.	Order is rejected.	Planned
TC-PUR-05	FR-PUR-005 / BR-011	Purchase total	Enter item quantities and unit costs.	Total equals derived line quantities × unit costs.	Planned
TC-PUR-07	FR-PUR-007 / BR-006	Receive purchase	Record valid received quantity.	Receipt succeeds and inventory increases by received quantity.	Planned
TC-PUR-08	FR-PUR-008	Over-receipt	Receive more than allowed ordered quantity.	Receipt is rejected unless approved exception exists.	Planned
TC-PUR-09	FR-PUR-009	Purchase history filter	Filter purchases by supplier/date.	Matching purchase records are shown.	Planned
TC-SALE-01	FR-SALE-001	Create sale	Create sale with one or more valid items.	Sale is accepted for authorized sales user.	Planned
TC-SALE-02	FR-SALE-002	Positive quantity	Enter zero/negative/non-integer quantity.	Invalid quantity is rejected.	Planned
TC-SALE-03	FR-SALE-003 / BR-008	Insufficient stock	Request quantity greater than available stock.	Sale is rejected; no completed sale is created.	Planned
TC-SALE-04	FR-SALE-004 / BR-011	Sale total	Enter valid quantities/prices.	Sale total equals derived line totals.	Planned
TC-SALE-05	FR-SALE-005	Decrease stock after sale	Complete valid sale.	Inventory decreases by sold quantity.	Planned
TC-SALE-06	FR-SALE-006	Unique transaction number	Complete multiple sales.	Each completed sale has a unique transaction identifier.	Planned
TC-SALE-07	FR-SALE-007	Sales history filter	Filter sales by transaction/date.	Matching sales are shown.	Planned
TC-REP-01	FR-REP-001	Inventory report	Open inventory report.	Product, SKU, category, quantity and stock status are shown.	Planned
TC-REP-02	FR-REP-002	Low-stock report	Open low-stock report.	Products at/below minimum stock are shown.	Planned
TC-REP-03	FR-REP-003	Purchase report	Filter purchase report by supplier/date.	Matching purchase data and totals are shown.	Planned
TC-REP-04	FR-REP-004	Sales report	Filter sales report by date.	Matching sales data and totals are shown.	Planned
TC-REP-05	FR-REP-005	Report consistency	Compare report totals with stored transactions.	Displayed totals agree with underlying stored transactions.	Planned
TC-COM-01	FR-COM-001	Required-field validation	Submit create/update form with required fields missing.	Operation is blocked with field-level validation.	Planned
TC-COM-02	FR-COM-002	Invalid input message	Enter invalid value in required field.	Clear validation message identifies the field/operation.	Planned
TC-COM-03	FR-COM-003	Success message	Complete valid create/update/receipt/adjustment/sale.	Confirmation message is displayed.	Planned
TC-COM-04	FR-COM-004	Empty-state search	Apply valid filter with no matches.	Explicit empty-state message is displayed.	Planned
TC-COM-05	FR-COM-005	Referential integrity	Deactivate referenced product/supplier.	References and historical records remain consistent.	Planned

12. Security Test Cases

Test ID	Requirement	Security Scenario	Procedure	Expected Result	Status
TC-SEC-01	SEC-001	Protected endpoint without authentication	Send request without valid authentication.	Protected operation is rejected.	Planned
TC-SEC-02	SEC-002	Password storage review	Inspect database/configuration representation of passwords.	Passwords are not stored in plaintext.	Planned
TC-SEC-03	SEC-003	Server-side authorization	Call admin-only operation as lower-privilege user.	Server rejects unauthorized operation.	Planned
TC-SEC-04	SEC-004	Client-side permission tampering	Modify client request/role information and retry restricted action.	Backend still enforces permissions.	Planned

TC-SEC-05	SEC-005	HTTPS production check	Inspect deployment/network behavior.	Credentials/session data use HTTPS in production.	Planned
TC-SEC-06	SEC-006	Input sanitization	Submit malformed/unexpected client values to API.	Backend validates/rejects unsafe or invalid input without unsafe DB behavior.	Planned
TC-SEC-07	SEC-007	Secret exposure review	Inspect frontend bundle/configuration/repository.	Database credentials and secrets are not exposed client-side.	Planned
TC-SEC-08	SEC-008	Account enumeration	Compare responses for unknown vs existing account with invalid credentials.	Failure behavior does not reveal account existence.	Planned
TC-SEC-09	SEC-009	Inactive account authentication	Authenticate using inactive account credentials.	Authentication fails.	Planned
TC-SEC-10	SEC-010	Least-privilege DB review	Inspect application database credentials/permissions.	Application account has only required database privileges.	Planned

13. Non-Functional Test Cases

Test ID	Requirement	Scenario	Procedure	Expected Result	Status
TC-NFR-01	NFR-001	API response time	Run defined normal authenticated API load and measure response times.	At least 95% of normal authenticated requests respond within 2 seconds under defined load.	Planned
TC-NFR-02	NFR-002	Availability review	Observe application during defined operating window excluding planned maintenance.	Availability target of at least 99% is demonstrated for the defined window.	Planned
TC-NFR-03	NFR-004	Duplicate retry	Retry a failed purchase/sale request.	Retry does not create duplicate transaction records.	Planned
TC-NFR-04	NFR-005	Maintainability review	Inspect module separation and core automated tests.	Business functionality is separated into identifiable modules with automated core-service tests.	Planned
TC-NFR-05	NFR-006	Testability review	Run deterministic API/service tests.	Core services behave deterministically for automated testing.	Planned
TC-NFR-06	NFR-007	Cross-browser	Execute core workflows in Chrome/Chromium, Firefox and Edge.	Web client works across required browsers.	Planned
TC-NFR-07	NFR-008	Data-volume test	Populate at least 10,000 products and 100,000 transaction/item records.	Schema supports target volume without redesign; key operations remain usable.	Planned
TC-NFR-08	NFR-009	Atomicity	Force failure during purchase/sale + inventory update.	No partial state remains; related updates commit or roll back together.	Planned
TC-NFR-09	NFR-010	Error-message usability	Trigger validation errors across major forms.	Message identifies field/operation needing correction.	Planned
TC-NFR-10	NFR-011	Code-quality review	Run agreed formatting, review and static-analysis checks.	Code follows agreed project quality practices.	Planned
TC-NFR-11	NFR-012	Auditability	Inspect inventory-affecting events.	Responsible user and timestamp are retained where applicable.	Planned

14. End-to-End Integration Scenarios

Scenario ID	Scenario	Flow	Expected Result
E2E-01	Purchase-to-stock	Create supplier → create purchase order → receive purchase → inspect inventory.	Stock increases only by the quantity actually received;

			inventory history links the change to the purchase.
E2E-02	Sale-to-stock	Create valid sale against available stock → inspect inventory/history.	Sale completes, unique transaction is created and stock decreases by sold quantity.
E2E-03	Insufficient-stock protection	Attempt sale above available quantity.	Sale is rejected and inventory remains unchanged.
E2E-04	Low-stock monitoring	Reduce stock through valid sale/adjustment until quantity ≤ minimum.	Product appears in low-stock identification/report.
E2E-05	Role enforcement	Attempt operations using users from different roles.	Only operations permitted by role succeed.
E2E-06	Reporting consistency	Perform purchase and sale transactions → generate reports.	Report values agree with stored transactions and current inventory.

15. Requirements Traceability Matrix

The SRS already contains a baseline RTM. The table below carries those selected mappings into the STP and adds the planned execution status. Test identifiers are planned identifiers until the corresponding test is executed.

Requirement	Brief Description	Design / Use Case	Planned Test
FR-AUTH-001	Valid login	UC-01 / Authentication	TC-AUTH-01
FR-AUTH-007	Backend authorization	AUTH-SEC / Permissions	TC-SEC-03
FR-PROD-001	Create product	PRODUCT-MOD / UC-03	TC-PROD-01
FR-PROD-002	Reject duplicate SKU	PRODUCT-MOD / validators	TC-PROD-02
FR-INV-001	Display stock	INVENTORY-MOD / UC-04	TC-INV-01
FR-INV-004	Low-stock identification	INVENTORY-MOD / UC-04/09	TC-INV-04
FR-PUR-003	Create purchase order	PURCHASE-MOD / UC-06	TC-PUR-03
FR-PUR-007	Increase stock on receipt	PURCHASE + INVENTORY / UC-07	TC-PUR-07
FR-SALE-003	Prevent sale above stock	SALES + INVENTORY / UC-08	TC-SALE-03
FR-SALE-005	Decrease stock after sale	SALES + INVENTORY / UC-08	TC-SALE-05
FR-REP-001	Inventory report	REPORT-MOD / UC-09	TC-REP-01
FR-REP-004	Sales report	REPORT-MOD / UC-09	TC-REP-04
FR-COM-001	Required-field validation	COMMON-MOD / validators	TC-COM-01
NFR-001	2-second API target	ARCH-PERF / API services	TC-NFR-01
NFR-009	Atomic updates	DB-ARCH / transaction services	TC-NFR-08
SEC-002	Password hashing	AUTH-SEC / auth configuration	TC-SEC-02
SEC-004	Block unauthorized API access	AUTH-SEC / permissions	TC-SEC-04
BR-003	Low-stock rule	INVENTORY-DOMAIN / rules	TC-INV-04
BR-008	Sale cannot exceed stock	SALES-DOMAIN / rules	TC-SALE-03

16. Test Schedule

Project Phase	Testing Activities	Output
Sprint 1 — Requirements & Foundation	Review testability of requirements; establish unit/API framework and test data approach.	Baseline test plan and initial test cases.
Sprint 2 — Products & Inventory	Unit/API tests for products, stock rules, adjustments and low-stock logic.	Product/inventory test results.
Sprint 3 — Suppliers & Purchases	Supplier, purchase order, receipt and inventory integration tests.	Purchase integration results.
Sprint 4 — Sales & Reporting	Sales, stock validation, reporting and filtering tests.	Sales/reporting results.
Sprint 5 — Integration & Quality	Regression, security, static analysis, performance and defect fixing.	Integrated test evidence.
Sprint 6 — Validation & Submission	System validation, RTM completion, final regression and documentation review.	Final test summary and submission package.

17. Test Evidence and Reporting

- Record test execution date, tester, build/commit, environment and pass/fail result.
- Attach screenshots, API responses, logs or database evidence for important failures and requirement demonstrations.
- Keep the final test cases and evidence under the team's Git repository alongside the SRS/SAD/STP.
- Update the RTM with actual implementation paths, test execution identifiers and final results after development.
- Summarize open defects and their disposition before final submission.

18. STP Conclusion

The STP provides functional, security, integration and non-functional coverage for the requirements baseline. The strongest integration focus is the correctness of inventory-changing transactions: receiving purchases must increase stock correctly, sales must decrease stock without permitting negative inventory, and the resulting events must remain traceable. The plan also explicitly tests the backend authorization and security requirements defined by the SRS.